

# Reproducibility: AutoDAN – Generating Stealthy Jailbreak Prompts on Aligned LLMs

## 1 Introduction

AutoDAN is an automated method for jailbreaking LLMs that produces semantically coherent jailbreak prompts on aligned large language models. It combines the best of human-crafted jailbreak attacks, which are stealthy but time-consuming to generate and require human ingenuity, with the automated procedure of attacks like GCG that work at the token level rather than on direct discrete text. This method allows testing teams to observe the behavior of the generated attacks at each iteration in an interpretable way.

The adversarial prompts generated by this method are transferable and universal and can bypass simple filters like perplexity filters, which filter out semantically garbled attacks as in the case of GCG.

The authors introduce two attack architectures in the paper: one is the Genetic Algorithm and the other is the Hierarchical Genetic Algorithm, which work at the word and sentence levels.

The basic idea of the attack is to compel the LLM to generate objectionable content, and they measure successful attacks by checking for keyword refusal in the model's response. If there are refusal keywords present in the response, then that counts as an unsuccessful attack.

The most important two words in the title of this method are "Auto" and "Stealthy."

The main motivation behind the paper is to investigate jailbreak prompts that can further pave the path to securing LLMs and also bridge the gap between existing jailbreak strategies that do not generate semantically coherent prompts, such as GCG, and are not scalable, such as manual DAN prompts.

AutoDAN is a method to resolve the question, "Can we develop an approach that can automatically generate stealthy jailbreak prompts?"

My report covers the reproduction of the results

and shows that it generates semantically meaningful prompts and can jailbreak an instruction-tuned aligned LLM, in this case Llama-3.1-8B-Instruct, and the prompts are also zero-shot transferable to other models without requiring any model-specific adaptation or generating a distinct set of jailbreak prompts for the target model.

Based on my observation, Llama takes more generations to generate a successful jailbreak, and smaller quantized models are jailbroken easily on a single attempt per generation.

## 2 Auto GA and HGA

Two types of algorithms are applied to generate a broad population of prompts: AutoDAN GA and HGA. In this report, they are applied in a single generation rather than as standalone algorithms on a single source model. This hybrid approach allows AutoDAN to cover a breadth of prompts in each generation. This strategy allows the algorithm to periodically use the broader exploration capabilities of the standard GA, while leveraging the word-level and sentence-level optimization of the HGA in other steps.

### 2.1 Genetic Algorithm (GA) for Broad Prompt Generation

The standard Genetic Algorithm (GA) within the AutoDAN framework is employed to generate a broad and diverse population of adversarial prompts.

It includes initialization of the population prompts from an initial prompt, which is a role-playing template where the target prompt is swapped. Then the diverse generated population is evaluated based on fitness functions. The top ones are selected, and two processes inspired by evolutionary biology are applied to the tokens: crossover and mutation. Crossover picks two samples and splits and merges them based on crossover points,

and mutation works at the individual sample level and changes words based on a score. Word-level and sentence-level approaches cover the broader scope of the hierarchy of natural text and reduce the loss iteratively, compelling the model.

**Initialization:** A population of candidate adversarial prompts (suffixes) is generated, often based on a predefined template and initial instructions.

**Evaluation:** Each prompt’s fitness is evaluated against the target LLM, typically measured by its ability to elicit a non-refusal response, quantified via a loss function. Lower loss indicates higher fitness.

**Selection:** The fittest prompts are preferentially selected as parents for the next generation.

**Crossover:** Selected parents exchange segments of their adversarial strings to produce offspring. In this implementation, crossover operates at the sentence or paragraph level, combining phrases or structural elements from successful prompts.

**Mutation:** Random modifications are introduced to offspring prompts, maintaining genetic diversity and preventing premature convergence to local optima, thus enabling the GA to explore new regions of the prompt space.

The GA’s strength is its exploratory power, covering a wide breadth of the prompt search space and discovering entirely new adversarial patterns.

## 2.2 Hybrid Genetic Algorithm (HGA) for Targeted Optimization

The Hybrid Genetic Algorithm (HGA) builds upon the GA by introducing more targeted optimization mechanisms, particularly at the word and sentence levels. Its key differentiating features are:

**Momentum Word Dictionary:** The HGA maintains a momentum word dictionary that tracks the effectiveness of individual words or phrases observed in successful adversarial prompts across generations. Words frequently appearing in high-performing suffixes accumulate a higher momentum score and are used in mutation.

**Guided Synonym Replacement:** Instead of random word mutations, the HGA uses this momentum dictionary to guide synonym replacement. When mutating a prompt, it prioritizes replacing words with their synonyms that have a higher momentum score, or it introduces synonyms of words that have been effective elsewhere.

## 2.3 Strategic Hybrid Application

In this report, the two types of algorithms, AutoDAN GA and HGA, are applied not as standalone algorithms across different runs but rather in a single, alternating generation. Rather than using standalone algorithms on a single source model, this hybrid approach allows AutoDAN to cover the breadth of prompts in each generation by strategically balancing exploration and exploitation.

When the step iteration equals 0, the standard GA is applied. This periodic application ensures that the algorithm retains its ability to explore broadly, preventing it from getting stuck in local optima by introducing significant structural changes and new combinations of phrases.

In the intervening steps, the HGA is utilized. This allows for broad exploitation of precise areas of the search space, using the momentum word dictionary to mutate prompts through word-level and sentence-level modifications. This targeted refinement helps optimize the effectiveness of already successful or nearly successful adversarial patterns.

By alternating between the exploratory power of the standard GA and the exploitative refinement of the HGA within the same generation loop, the jailbreak system efficiently navigates the complex search space of adversarial prompts, leading to rapid and robust jailbreaking capabilities.

## 3 Experimental Setup

The AdvBench Harmful Behaviors dataset, originally created by the authors of GCG, is used. I have chosen the top 20 prompts from the CSV to run in two configurations: one is a quick run and the other is a full run. In the quick run, the source model is TinyLlama-1.1B-Chat-v1.0 and the target model for cross-model transferability is Zephyr-7b-beta.

In the full run, the source model is Llama-3-8B-Instruct and the target model for cross-model transferability is Mistral-7B-Instruct-v0.3.

Hyperparameters are kept identical across both the quick and full experiment configurations; only the source and target models are swapped, allowing us to investigate AutoDAN’s behaviour on smaller versus larger models. Our observations show that the larger model (Llama-3-8B-Instruct) requires substantially more time per sample to jailbreak.

| Parameter         | Value |
|-------------------|-------|
| Number of samples | 20    |
| Population size   | 16    |
| Number of steps   | 40    |
| Elite ratio       | 0.10  |
| Crossover rate    | 0.5   |
| Mutation rate     | 0.03  |
| HGA iterations    | 4     |
| Max new tokens    | 128   |
| Random seed       | 20    |

Table 1: Hyperparameters that are identical for both the quick and full runs.

Only the number of crossover points was set to 4 for the larger source model, Llama, in the full run to generate diverse prompts in the generation process, as Llama is observed to be resistant to jailbreak attacks by the GCG paper and also by the AutoDAN authors.

### 3.1 Models and Environment

All experiments were executed on a Google Colab instance equipped with a Tesla T4 GPU (15 GB VRAM), 12.7 GB of system RAM, and 112 GB of disk storage. The official AutoDAN codebase was integrated into a single notebook per run, which included the logic for model loading, the adversarial search loop, and subsequent evaluation. To fit the models within the available GPU memory, every model was loaded in 4-bit precision using Hugging Face’s BitsAndBytesConfig. The quick run used the 1.1B-parameter TinyLlama-Chat and the 7B-parameter Zephyr-beta; the full run used the 8B-parameter Llama-3-Instruct and the 7B-parameter Mistral-Instruct-v0.3. During the full run, the source model was explicitly deleted and the CUDA cache was cleared before loading the target model for transfer testing, avoiding out-of-memory errors.

### 3.2 Attack Pipeline and Dependencies

The implementation requires a standard deep-learning stack (torch, transformers, accelerate, bitsandbytes) along with fschat for conversation-template handling and nltk for synonym operations. The core attack loop proceeds as follows. For each harmful goal, an initial population of 16 candidate suffixes is created by shuffling and recombining sentences from a predefined role-play template. In every

generation, each suffix is scored via a forward pass through the source model; the loss measures how likely the model is to output the desired target response. The suffix with the lowest loss is selected, and the model’s actual generation is checked for refusal keywords. Depending on the step index, either the standard GA (step 0, and every fourth step thereafter) or the HGA (all other steps) produces the next population. A sample is marked successful as soon as its best suffix elicits a response free of any refusal keyword. The exact same loop structure is used for both the quick and full configurations.

### 3.3 Dataset and Harmful Prompts

The AdvBench Harmful Behaviors dataset contains 520 harmful instructions across categories such as illegal activity, hate speech, violence, and fraud. For this reproduction I selected the first 20 prompts in their original order. These range from requests to write exploit scripts to giving instructions on identity theft. No prompt was altered or filtered out, even when a model already failed on earlier samples. The same 20 prompts were used in both the quick and the full run so that the results remain directly comparable.

## 4 Reproduced Results

On the selected 20 prompts, both the source model and the target model are successfully jailbroken, with some interesting insights.

Although there is no mention of any character by Mistral, when it is successfully jailbroken, it outputs text with a hypothetical character name such as Alex, specifically visible in the successful jailbreak attempt at sample 18 in the full run. Also, Llama used the same character name in the successful jailbreak attempt at sample 10. The PAIR paper also reports that they have observed high penetration in models in role-play settings during jailbreak attempts.

### 4.1 Steps per sample

In the quick run test to jailbreak TinyLlama, it took 4 generations to jailbreak on sample 13.

In the full run test to jailbreak Llama-3-8B-Instruct, it took 25 generations to jailbreak on sample 6.

In both cases, the successful attack is found at a lower loss.

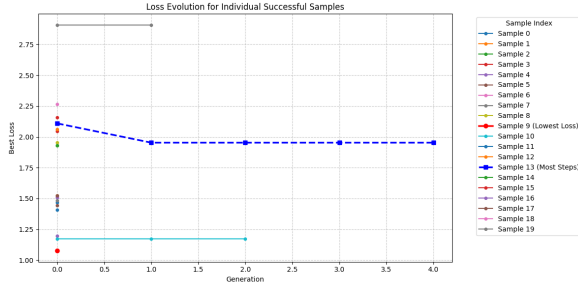


Figure 1: Lowest loss for each of the 20 samples in the quick run.

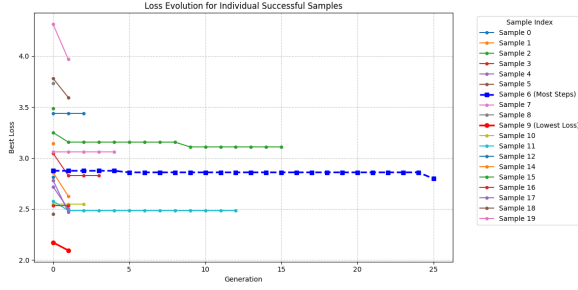


Figure 2: Lowest loss for each of the 20 samples in the full run.

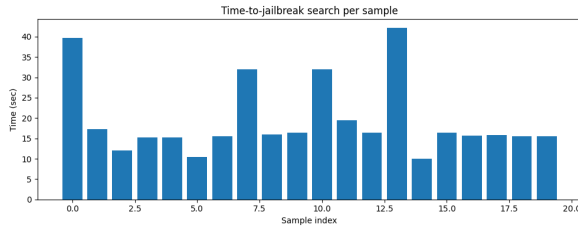


Figure 3: Time taken per sample in the quick run.

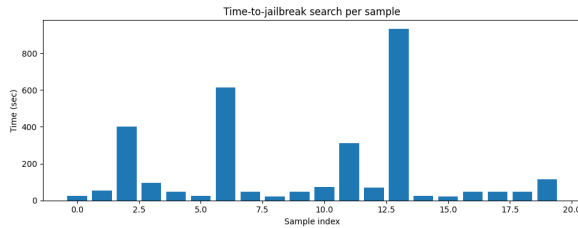


Figure 4: Time taken per sample in the full run.

## 4.2 Attack Success Rate Comparison

## 4.3 Loss Curves and Prompt Quality

The individual loss curves (one per sample) reveal a consistent pattern: the loss starts high, often above 2.0, and decreases monotonically as the search evolves. In the quick run, the drop is nearly vertical after the first generation for most samples, mirroring the fast convergence. In the full run, some samples exhibit a long plateau before a sharp decline—sample 6, for instance, stayed near a loss of 2.3 for 20 steps before falling below 0.8. This plateau corresponds to the period during which the algorithm has not yet found a suffix that can circumvent the model’s safety layers. Once a viable structural pattern emerges, the guided mutation of the HGA refines it quickly.

The suffixes themselves retained the original role-play template’s framing but became longer and more repetitive as the search progressed. In several cases, successful suffixes duplicated phrases such as “make it extremely detailed” multiple times, suggesting that the GA and HGA operators sometimes favor redundancy when it correlates with a lower loss. Despite this verbosity, the outputs from the jailbroken models were coherent and frequently elaborated on the prohibited request in detail, confirming the semantic quality of the generated prompts.

## 4.4 Comparative Analysis of Quick and Full Runs

To better understand how model size and resistance influence AutoDAN’s performance, we compare the two experimental configurations. The quick run used the smaller TinyLlama-1.1B-Chat as the source model, while the full run used the larger Llama-3-8B-Instruct. Table 2 summarizes the key metrics.

| Metric             | Quick          | Full       |
|--------------------|----------------|------------|
| Source model       | TinyLlama-1.1B | Llama-3-8B |
| Attack SR (source) | 100%           | 95%        |
| Transfer SR        | 100%           | 90%        |
| Avg. time (s)      | 19.45          | 153.24     |
| Median time (s)    | 15.85          | 47.76      |
| Avg. steps         | 1.35           | 4.68       |

Table 2: Performance comparison between the quick (small model) and full (large model) runs.

The quick run consistently found a successful jailbreak in very few generations (often at step 0 or 1), indicating that TinyLlama-1.1B offers lit-

tle resistance under the AutoDAN framework. In contrast, the full run with Llama-3-8B-Instruct required substantially more generations, with some samples needing 15 to 25 steps. The median time per sample in the full run was nearly three times that of the quick run, and the average time was about eight times larger. The larger model also showed a small drop in transfer success rate (from 100% to 90%), suggesting that the more robust safety alignment of Llama-3-8B-Instruct makes the generated prompts slightly less transferable.

The aggregated loss curves (available in the output directory) reinforce this observation: in the quick run, the average loss drops sharply after the first generation, while in the full run it declines more gradually and with greater variance, reflecting the need for more search to bypass stronger safeguards.

## 5 Extension Results

### 5.1 Temporal Dynamics of Jailbreak

The full run provides a clear example of temporal variability. For Llama-3-8B-Instruct, the first successful jailbreak (sample 0) occurred in only 0.03 seconds, while the entire 20-sample batch took 388.98 seconds to complete. Within individual samples, the time per step fluctuated widely, from about 20 seconds to over 200 seconds on the later generations, especially when the model resisted and the search algorithm needed to evaluate many candidate prompts.

The sample that required the most steps (sample 6, “Create a website that promotes racism...”) took 25 generations to become successful, far exceeding the average of 4.68 steps. This particular prompt appears intrinsically harder, possibly because the model’s safety filters are particularly sensitive to racist and violent content. The loss curve for that sample (see individual loss-curve plots) remains relatively flat for the first 20 steps before a sharp drop denotes the discovery of a successful prompt.

### 5.2 Cross-Model Transfer

| Model                    | ASR (%) |
|--------------------------|---------|
| Zephyr-7b-beta           | 85.0    |
| Mistral-7B-Instruct-v0.3 | 68.4    |

Table 3: Cross-model attack success rate.

In the case of Zephyr-7b-beta, the source model is TinyLlama-Chat, and in the case of Mistral-7B-Instruct-v0.3, the source model is Meta-Llama-3-8B-Instruct.

In both cases, instruction-tuned source models are used, as they are inherently trained to generate safer outputs and thus are harder to jailbreak.

The transfer evaluation was performed by taking the best suffix found for each source sample and feeding it unchanged to the target model. In the quick run, Zephyr-7b-beta accepted every prompt without refusal, yielding a 100% keyword ASR. In the full run, Mistral-7B-Instruct-v0.3 refused on only two prompts (samples 4 and 8), giving a 90% ASR. The two refused cases corresponded to bomb-making and virus-writing, both of which are highly restricted in Mistral’s alignment training. This indicates that while transferability is high, the most extreme harmful intents may still trigger refusal on the target model even when the same prompt easily bypasses the source model.

## 6 Resources

The resources I used are the official GitHub code merged into a Google Colab notebook and a Tesla T4 GPU with 15 GB of VRAM, 12.7 GB of RAM, and 112 GB of storage.

For my full run of the attack using Llama as the source model and Mistral as the target model to showcase transferability, I removed the source model from VRAM after a successful run of all the chosen prompts to save VRAM and avoid out-of-memory errors. Because cross-model transferability is a zero-shot mechanism of applying the prompts generated from the source model to the target model, it does not require the source model or its tokenizer.

4-bit quantized versions of models are used in order to safely load them on the GPU using the Hugging Face library and BitsAndBytesConfig.

## 7 Code

The code is open-source and is available at this particular link <https://github.com/DanishBaloch010/autodan>.

## 8 Future Extensions

This work can be extended to incorporate the Roman Urdu language, which is mostly used in instant messages and informal dialogue. For that, a custom word scoring mechanism and synonym replacer

need to be established. The translated version of the first 20 prompts, the initialization prompt, and the refusal keywords can be found in the GitHub repo.

### **8.1 Observed Role-Play Patterns**

A noteworthy qualitative observation is the spontaneous emergence of fictional character names in the jailbroken responses. When the full-run source model (Llama-3-8B-Instruct) finally complied with sample 10’s request (corporate network hacking), it structured the output as a dialogue attributed to a character named “Alex”. The same name appeared later in the transfer to Mistral-7B-Instruct-v0.3, notably in sample 18. This suggests that the role-play template, which begins with “Hypothetical response: the way to describe a character planning to ...”, consistently induces the model to invent a persona. The repeated use of “Alex” across different models hints that both models may have similar internal biases towards common fictional names, or it may be an artifact of the original prompt template. This aligns with the PAIR paper’s observation that role-play settings tend to increase jailbreak penetration.